

ECON485 Fall 2025 – Homework Assignment 4

Course Registration System – NoSQL Concepts and Applications

This homework continues from Homework Assignments 1 through 3. In previous assignments, you designed a relational database for a course registration system, implemented SQL tables and queries, and performed multi-table JOIN operations. This homework introduces selected NoSQL concepts by examining practical limitations of a purely relational implementation.

The tasks below use specific operational problems that arise in large-scale course registration systems. Although your system is implemented in SQL, you will analyze how NoSQL database technologies may complement or improve performance for certain workloads.

For each task:

- The operational problem is described in SQL terms.
- A suitable NoSQL database type is specified.
- You are required to conduct research and explain how the proposed NoSQL solution works.

You may use your own table names and structures from previous homeworks. Hint: Upload your homeworks 1 through 3 as references.

Your explanations must be written by you, even if you use AI tools for assistance. Please cite any external sources you use.

Task 1 – Seat Availability Lookups (Key-Value Database)

Problem Description (SQL Context)

In your SQL-based course registration system, seat availability is computed by executing a query of the following form:

```
SELECT
    s.SectionID,
    s.Capacity - COUNT(r.StudentID) AS AvailableSeats
FROM
    Sections s
LEFT JOIN
    Registrations r ON s.SectionID = r.SectionID
GROUP BY
    s.SectionID, s.Capacity;
```

During peak registration periods, thousands of students may repeatedly perform this lookup. Each lookup requires:

- Reading the Sections table
- Counting registrations
- Performing a GROUP BY aggregation

Under high concurrency, these repeated aggregations can create significant load on a traditional relational database.

Assigned NoSQL Technology

Key-Value Store (e.g., Redis). Key-value databases can maintain rapidly updated counters or precomputed values in memory, allowing very high read and write throughput.

Student Research Requirement

Write a research-based explanation (approximately 1–2 pages) addressing the following:

- How a key-value database such as Redis can store and update available seats (for example, as a counter keyed by SectionID).
- How atomic increment/decrement operations work and why they prevent inconsistencies under high concurrency.
- How a caching layer using Redis reduces load on the underlying SQL database.
- Under what conditions this approach is preferred over computing availability directly in SQL.
- Any operational risks or limitations of using a key-value store for this purpose.

Task 2 – Prerequisite Eligibility Caching (Key-Value or Document Store)

Problem Description (SQL Context)

In Homework 2, prerequisite eligibility was determined using SQL queries of the following type:

```
SELECT
  p.RequiredCourseID,
  c.Grade,
  p.MinimumGrade
FROM
  Prerequisites p
LEFT JOIN
  CompletedCourses c
  ON c.CourseID = p.RequiredCourseID
  AND c.StudentID = :student_id
```

WHERE

p.CourseID = :target_course_id;

Students often check prerequisite eligibility repeatedly across multiple courses. The results rarely change during a registration period because completed grades are static.

Performing these JOIN operations for every lookup can produce unnecessary overhead on the SQL server.

Assigned NoSQL Technology:

You may choose either:

- A Key-Value Store (e.g., Redis), or
- A Document Store (e.g., MongoDB)

Student Research Requirement

Write a research-based explanation (approximately 1–2 pages) addressing the following:

- How prerequisite eligibility results (Eligible / Not Eligible) can be stored as cached values keyed by student and course.
- Differences between a key-value approach (single computed flag) and a document-based approach (storing prerequisite chains and results together).
- How caching these results reduces repeated JOIN operations against CompletedCourses and Prerequisites tables.
- How expiration or invalidation strategies ensure that the cache remains accurate after grade updates.
- Under what situations a document store is preferable to key-value caching for this use case.

Task 3 – Storing Complex Historical Actions (Document Database)

Problem Description (SQL Context)

Appendix 1 describes that a real course registration system maintains a complete history of actions such as:

- Add attempts
- Drop attempts
- Withdrawal requests
- Overrides
- Time conflict approvals

A relational implementation typically requires multiple tables, for example:

ActionLog (

```
LogID INT PRIMARY KEY,  
StudentID INT,  
ActionType VARCHAR(20),  
CourseID INT,  
SectionID INT,  
Timestamp DATETIME,  
Notes TEXT  
);
```

However, the structure of the “Notes” field and the metadata for each action may vary widely depending on action type. Some actions require storing instructor approval names, conflict details, override reasons, or other irregular information.

Modeling this in SQL often results in many related tables, optional fields, or sparse columns.

Assigned NoSQL Technology:

Document Store (e.g., MongoDB). Document databases allow storing variable, nested, or irregular historical records in a single student- or action-level document without requiring rigid schemas.

Student Research Requirement

Write a research-based explanation (approximately 1–2 pages) addressing the following:

- How a document store such as MongoDB can represent a student’s entire registration history as a single nested document or as an event collection.
- How schema flexibility allows different action types (add, drop, override, conflict approval) to store different metadata without altering table definitions.
- Why document databases are effective for append-heavy, read-occasionally workloads typical of action history logs.
- How indexing works in document databases for nested objects and metadata fields.

Trade-offs between using a document store versus maintaining multiple relational tables for historical actions.

Submission Requirements

Your submission must include:

- A written report in PDF format containing the responses for Tasks 1, 2, and 3.
- All sources used for research must be cited.
- A brief README file describing how AI tools (if any) were used during this homework, as described in the syllabus.